instituto superior de engenharia de lisboa

Departamento de Engenharia Informática

U!REKA - Play4Change: A Multiplatform Gamified Educational Platform for Sustainability and Digital Education

51620 : Radesh Ilesh Gamanbhai Govind (A51620@alunos.isel.pt)

Relatório para a Unidade Curricular de Licenciatura em Engenharia Informática e de Computadores da Licenciatura em Engenharia Informática e de Computadores

Professores : Professor Nuno Miguel Soares Datia
Professor António Serrador

21 – março – 2026

Resumo

U!REKA Play4Change é uma plataforma educacional gamificada e multiplataforma concebida para promover o envolvimento sustentado com temas de sustentabilidade e educação digital, desenvolvida como projeto de final de licenciatura no ISEL. A plataforma resolve dois problemas estruturais comuns ao software educativo existente: o estrangulamento na autoria manual de conteúdos, em que o esforço humano cresce proporcionalmente com a escala da plataforma, e a fragmentação dos ecossistemas móveis, em que manter aplicações nativas separadas para Android e iOS duplica o custo de engenharia. Ambos os problemas são resolvidos simultaneamente pela mesma base arquitetural.

O cliente móvel é construído com KMP (Kotlin Multiplatform) e Compose Multiplatform, disponibilizando uma aplicação Android completamente funcional e um alvo iOS a partir de uma única base de código partilhada. Novas tarefas educativas são produzidas automaticamente por um processo em lote noturno que invoca a API do modelo de linguagem Mistral através da biblioteca LangChain4j. Executar a geração num horário fixo, em vez de a pedido do utilizador, elimina a latência de inferência do caminho de pedido e desacopla completamente a disponibilidade da plataforma da disponibilidade do serviço de inteligência artificial. A repetição de conteúdos ao longo dos dias é evitada através da desduplicação semântica: cada tarefa gerada é representada como um vetor numérico de elevada dimensão e comparada com todos os vetores previamente armazenados usando similaridade de cosseno dentro do PostgreSQL com a extensão pgvector, descartando tarefas cujo significado é demasiado semelhante ao de conteúdos existentes, independentemente das diferenças de formulação.

A identidade dos jogadores é gerida através de autenticação delegada por OAuth 2.0 combinada com uma arquitetura de sessão de dois tokens: um token de acesso de curta duração mantido apenas em memória e um token de atualização de longa duração rotacionado em cada utilização, permitindo a deteção automática de credenciais roubadas. Um motor de pontuação do lado do servidor atribui recompensas configuráveis por conclusões corretas e atempadas e penalizações por inatividade. O backend é executado em Spring Boot e implantado como uma aplicação multi-contentor via Docker Compose. O modelo de custo em lote escala com o número de domínios temáticos em vez do número de utilizadores, tornando a plataforma progressivamente mais eficiente em termos de custo à medida que a adoção cresce.

Abstract

U!REKA Play4Change is a multiplatform gamified educational platform designed to promote sustained engagement with sustainability and digital education, developed as a final year undergraduate project at ISEL. The platform addresses two structural problems common to existing educational software: the manual content authoring bottleneck, where human effort must scale proportionally as the platform grows, and mobile platform fragmentation, where maintaining separate native applications for Android and iOS doubles engineering cost. Both problems are resolved simultaneously by the same architectural foundation.

The mobile client is built with KMP (Kotlin Multiplatform) and Compose Multiplatform, delivering a fully functional Android application and an iOS target from a single shared codebase. New educational tasks are produced automatically by a nightly batch pipeline that invokes the Mistral language model API through the LangChain4j library. Executing generation at a scheduled time rather than on user request eliminates inference latency from the user-facing request path and decouples platform availability from AI service availability entirely. Content repetition across days is prevented through semantic deduplication: each generated task is embedded as a high-dimensional numerical vector and compared against all previously stored vectors using cosine similarity within PostgreSQL extended with the pgvector module, discarding tasks whose meaning is too similar to existing content regardless of phrasing differences.

Player identity is managed through OAuth 2.0 delegated authentication combined with a dual-token session architecture: a short-lived access token held only in application memory and a long-lived refresh token rotated on every use, enabling automatic detection of stolen credentials. A server-side scoring engine assigns configurable rewards for timely correct completions and penalties for inactivity, sustaining the daily engagement loop. The backend runs on Spring Boot and is deployed as a multi-container application via Docker Compose. The batch cost model scales with the number of subject domains rather than user count, making the platform progressively more cost-efficient as adoption grows.

Índice

Lista de Tabelas **xi**

1 Introduction **1**

1.1 Context and Motivation 1

1.2 Problem Statement 2

1.3 Proposed Solution 2

1.4 Objectives 3

1.5 Document Structure 3

2 State of the Art **5**

2.1 Gamification in Education 5

2.1.1 Evidence of Effectiveness 5

2.2 Serious Games 6

2.3 Multiplatform Mobile Development 6

2.3.1 Native versus Cross-Platform Approaches 6

2.3.2 Justification for Kotlin Multiplatform 7

2.4 AI-Generated Content in Education 7

2.4.1 Large Language Models for Educational Content 7

2.4.2 LangChain4j and the Mistral API 7

2.4.3 Semantic Deduplication with pgvector 8

2.5 Related Platforms 8

2.6 Identified Gap 9

3 Architecture and Design **11**

3.1 Architectural Overview 11

3.2 Content Generation Pipeline 11

3.2.1 The Case Against On-Demand Generation 11

3.2.2 Batch Pre-Generation . . . 12
3.2.3 Semantic Deduplication . . . 13
3.2.4 Content Quality Validation . . . 14
3.3 Data Persistence . . . 14
3.3.1 Why a Single Relational Database . . . 14
3.3.2 Transactional Correctness and ACID . . . 15
3.3.3 Concurrent Write Safety . . . 15
3.4 Mobile Client Architecture . . . 16
3.4.1 Offline-First Design . . . 16
3.4.2 Caching Strategy . . . 16
3.5 Authentication and Session Management . . . 17
3.5.1 Delegated Authentication . . . 17
3.5.2 Dual-Token Architecture . . . 17
3.6 Scoring Engine . . . 18
3.7 Consistency Trade-offs . . . 18
3.7.1 Task Content: Availability over Consistency . . . 18
3.7.2 Player Scores: Consistency over Availability . . . 18
3.8 Observability . . . 19
3.9 Deployment . . . 19

4 Implementation **21**
4.1 Monorepo Structure . . . 21
4.2 Kotlin Multiplatform Client . . . 21
4.2.1 Shared Business Logic . . . 21
4.2.2 Compose Multiplatform UI . . . 22
4.2.3 Navigation with Decompose . . . 22
4.2.4 Dependency Injection with Koin . . . 22
4.3 Backend: Spring Boot . . . 22
4.3.1 REST API Design . . . 22
4.3.2 Nightly Batch Job . . . 23
4.3.3 Scoring Engine Implementation . . . 23
4.4 Content Generation Pipeline . . . 23
4.4.1 LangChain4j Integration . . . 23

4.4.2 Mistral API Configuration 24
4.4.3 Schema Validation and Retry Logic 24
4.4.4 pgvector Deduplication 24
4.5 Database Schema 24
4.6 Authentication Implementation 25
4.7 Deployment with Docker Compose 25

5 Evaluation **27**
5.1 Functional Testing 27
5.2 Performance Evaluation 28
5.2.1 Batch Generation Latency 28
5.2.2 API Response Latency 28
5.2.3 Cold-Start Behaviour 28
5.3 Cost Model Validation 28
5.4 Limitations and Future Work 29
5.4.1 Absence of Per-User Personalisation 29
5.4.2 iOS Deployment 29
5.4.3 Observability Dashboard 29

6 Conclusion **31**
6.1 Summary of Contributions 31
6.2 Objectives Achieved 31
6.3 Reflection on Engineering Decisions 32
6.4 Future Work 33

Referências **35**

A Applied Survival Analysis by Hosmer and Lemeshow **i**

Lista de Tabelas

3.1 Comparison of content generation strategies. 12

5.1 Daily AI generation cost comparison: batch versus on-demand. 29

Capítulo 1

Introduction

1.1 Context and Motivation

Education systems worldwide face a persistent challenge: sustaining learner engagement over time, particularly for subjects such as sustainability and digital literacy that require continuous effort rather than a single study session. Unlike topics learned in a short burst, these disciplines ask learners to build habits and update knowledge regularly. Traditional instructional platforms respond to this challenge with static question banks authored by human experts. The problem is that manual authoring is expensive: producing one high-quality educational task may take an experienced author between thirty minutes and several hours. As a platform grows to cover more subjects, more difficulty levels, or more languages, the authoring workforce must grow proportionally. The economics do not improve with scale.

Gamified platforms have emerged as a structural response to the engagement problem. Research consistently shows that game mechanics — points, streaks, leaderboards, and immediate feedback — produce measurable improvements in learner persistence and knowledge retention [2]. A player who earns points for completing today's task has a reason to return tomorrow. A player whose streak is at risk of a penalty has a reason not to skip a day. These mechanisms convert educational participation from an act of willpower into a structured behaviour reinforced by the platform itself.

However, two structural problems remain unsolved in combination by existing platforms. First, the content authoring bottleneck: even the most successful gamified platforms, such as Duolingo, depend on large teams of human authors to produce and maintain content. Automated content generation using AI models offers a path to removing this bottleneck, but doing so requires careful engineering — AI inference is slow, expensive, and non-deterministic, and cannot simply be placed on the path of a user's request. Second, the mobile platform fragmentation problem: the two dominant mobile ecosystems, Android and iOS, use different programming languages and toolchains. A team that builds and maintains two separate codebases doubles engineering cost, doubles the time required to ship any feature, and doubles the surface area for bugs. Cross-platform development frameworks have matured considerably in recent years, but their systematic adoption in gamified educational platforms remains limited.

U!REKA Play4Change was designed to address both problems simultaneously. The platform produces educational tasks automatically, using a scheduled AI content generation pipeline that runs nightly and stores results before users ever request them. It delivers an Android application and an iOS application

from a single shared codebase, using KMP (Kotlin Multiplatform) — a framework that allows the same business logic and user interface code to run on both platforms. The result is a platform that scales content without scaling authoring effort, and scales across mobile platforms without scaling the engineering team.

1.2 Problem Statement

Existing gamified educational platforms cannot scale content production without proportionally scaling human authoring effort. The platforms that have succeeded in sustaining engagement over time — Duolingo being the clearest example — do so through the work of large, specialised content teams. This model is not accessible to small educational institutions, individual researchers, or non-profit organisations with limited resources. Automated content generation using large language models (LLMs — AI systems trained to produce coherent text from instructions) offers a path to removing this constraint, but introduces its own engineering challenges: inference latency, availability dependency on third-party AI services, cost that scales with users, and the risk of generating repetitive content over time.

Simultaneously, building native applications for both Android and iOS requires separate development teams, separate codebases, and separate testing and maintenance cycles. For organisations without the resources of large technology companies, this fragmentation is a practical barrier to multiplatform deployment. Current cross-platform solutions each make different trade-offs between code sharing, performance, and platform fidelity. None has been adopted by a production gamified educational platform that also solves the AI content generation problem.

No existing platform solves both problems in combination. U!REKA Play4Change addresses this gap.

1.3 Proposed Solution

U!REKA Play4Change is a gamified educational platform built on four interconnected engineering components.

The first is a single Kotlin Multiplatform codebase delivering both Android and iOS applications. Shared business logic — the scoring engine, data models, network clients, and local persistence — is written once and compiled for both platforms. The shared declarative user interface, built with Compose Multiplatform, renders natively on each platform without visual compromise.

The second is a nightly AI content generation pipeline. A scheduled batch job runs each night, invoking the Mistral language model API through the LangChain4j library to generate the educational tasks for the following day. By separating the moment of generation from the moment of delivery, the platform avoids placing AI inference latency on the user's request path. Content is ready in storage when users arrive.

The third is semantic deduplication using pgvector (a vector similarity search extension for the PostgreSQL database). Because language models produce different phrasings of the same concept across separate invocations, exact text comparison cannot detect repetition. Each generated task is converted to a numerical vector — a representation of its meaning rather than its words — and compared against all previously stored vectors. Tasks that are conceptually too similar to existing content are discarded before storage.

The fourth is a server-side scoring engine that awards points for correct and timely task completions, applies penalties for inactivity, and manages badge awards. The scoring logic lives on the server, not the client, to prevent manipulation and to ensure correctness under concurrent submissions.

The backend runs on Spring Boot (a Java framework for building server applications) and is deployed as a set of coordinated containers using Docker Compose (a tool that manages multiple application services as a single unit).

1.4 Objectives

The project pursued four core engineering objectives.

Objective 1: Multiplatform delivery from a single codebase. Deliver a functional Android application and an iOS application from a single Kotlin Multiplatform project, sharing all business logic and as much user interface code as the platform allows, without sacrificing native look, feel, or performance.

Objective 2: Adaptive AI-generated content with semantic deduplication. Implement a nightly batch pipeline that generates domain-specific educational tasks using the Mistral language model API, validates their structure, and filters out semantically duplicate content using vector embeddings and cosine similarity search via pgvector, so that players always receive fresh content.

Objective 3: Engagement loop engineering. Implement a server-side scoring engine with configurable reward and penalty thresholds, badge awards, and streak tracking, such that the platform produces the behavioural reinforcement patterns identified by gamification research as drivers of sustained engagement.

Objective 4: Cost efficiency through batch pre-generation. Demonstrate that the nightly batch model produces a fixed daily cost independent of user count, in contrast to the on-demand model whose daily cost scales linearly with active users, and validate this model at the scale of the platform's initial deployment.

1.5 Document Structure

The remainder of this report is organised as follows. Chapter 2 surveys the relevant literature and existing technologies, covering gamification in education, serious games, multiplatform mobile development frameworks, AI content generation, and related platforms, and identifies the gap that this project addresses. Chapter 3 describes the system architecture in depth, following the engineering reasoning behind each design decision: the content generation pipeline, data persistence, mobile client design, authentication, the scoring engine, consistency trade-offs, observability, and deployment. Chapter 4 presents the implementation, detailing how the architectural decisions were realised in code, including the monorepo structure, Kotlin Multiplatform client, Spring Boot backend, LangChain4j pipeline, database schema, and Docker Compose deployment. Chapter 5 covers evaluation, reporting on functional testing, performance measurements, and a cost model validation. Chapter 6 draws conclusions, reflects on the engineering process, and outlines directions for future work.

Capítulo 2

State of the Art

This chapter surveys the technical and research landscape relevant to U!REKA Play4Change. It covers gamification in education, serious games, multiplatform mobile development, AI-generated content, semantic deduplication, and related platforms. The chapter concludes by identifying the gap that the platform addresses.

2.1 Gamification in Education

Gamification is defined as the application of game design elements in non-game contexts [1]. The term distinguishes the application of specific game mechanics — points, leaderboards, badges, progress bars, and challenge structures — from full-scale game design. In educational settings, gamification is applied to existing content and activities with the goal of increasing learner motivation without changing the learning material itself.

The underlying motivation rests on a well-established observation in motivational psychology: game mechanics activate intrinsic drivers of engagement that are otherwise absent in purely didactic settings. Competition, mastery progression, and immediate reward all contribute to sustained voluntary effort. Gamification attempts to transfer these drivers into learning contexts where they do not arise naturally.

2.1.1 Evidence of Effectiveness

Empirical evidence on gamification's effectiveness in education has accumulated steadily. A systematic review of 24 empirical studies found positive effects in the majority of cases, with social comparison mechanisms such as leaderboards and peer challenges producing the most consistent engagement gains [2]. The review analysed studies across a range of learning contexts, student ages, and gamification element types, finding that point systems and progress indicators are the most consistently effective elements across diverse populations.

However, the same review noted important moderating factors: the specific gamification element employed, the duration of the intervention, and the demographic profile of learners all influence outcomes. Long-duration interventions show diminishing effects if the novelty of the game element wears off and is not replaced by genuine mastery progression. This finding informs the design of U!REKA Play4Change's scoring engine, which combines individual progression with configurable penalty and

reward thresholds rather than relying solely on competitive ranking, and sustains novelty through AI-generated content that changes daily.

2.2 Serious Games

Serious games are digital games designed with a primary purpose beyond pure entertainment, typically targeting learning, training, or behaviour change [7]. Unlike gamification, which adds game elements to an existing non-game activity, a serious game is a game in its own right — it has goals, rules, feedback mechanisms, and voluntary participation — but its design is oriented around a learning or behavioural objective.

A meta-analysis of serious game research spanning a decade found statistically significant improvements in learning outcomes across multiple disciplines, including science, health, and civic education [7]. The analysis identified two mechanisms as the strongest predictors of learning gain: immediate feedback after every action, and clear goal structures that give players an unambiguous sense of what they are working toward. Feedback enables learners to adjust understanding in real time; goal structure provides the motivational frame that makes continued effort rational.

U!REKA Play4Change draws directly on these findings. Each daily task is structured as a self-contained challenge with a defined correct response, delivering immediate scoring feedback. The daily goal — completing the day's task — is unambiguous. The streak and penalty system extends the goal structure across days, making each day's engagement part of a larger progression narrative.

2.3 Multiplatform Mobile Development

2.3.1 Native versus Cross-Platform Approaches

Mobile application development historically bifurcated into two models. The native model uses separate codebases per platform: Swift or Objective-C for iOS, and Java or Kotlin for Android. Each codebase is developed, tested, and maintained independently, with full access to each platform's APIs and the highest achievable performance. The cross-platform model uses a single codebase that targets multiple platforms through an abstraction layer, trading some degree of platform-specific control for reduced duplication.

The native model's engineering cost is its decisive weakness for small teams: any feature, bug fix, or design change must be implemented twice, in two different languages, and verified on two different platforms. At scale, this doubles the engineering resource required for any given throughput of feature delivery.

Established cross-platform frameworks each make different architectural choices. React Native compiles JavaScript logic to native platform components but routes all interaction through a JavaScript bridge, introducing latency that manifests in complex animated interfaces. Flutter compiles to native ARM code and provides its own rendering engine, achieving near-native performance but drawing all UI components itself rather than using platform-native widgets, which can diverge from each platform's visual

conventions. KMP (Kotlin Multiplatform) takes a fundamentally different approach: it shares only business logic across platforms while allowing the user interface layer to use either platform-native frameworks or the shared Compose Multiplatform declarative UI toolkit. This means KMP inherits native platform UI components where desired and shares UI code where appropriate, achieving a balance that neither React Native nor Flutter offers.

2.3.2 Justification for Kotlin Multiplatform

U!REKA Play4Change adopts KMP for shared business logic — including the scoring engine, data models, network clients, and local caching strategies — combined with Compose Multiplatform for the shared declarative user interface. This combination allows Android and iOS to share both logic and UI components from a single codebase while maintaining access to native platform APIs where required. The choice reduces duplication and accelerates feature delivery without sacrificing the performance characteristics required for smooth animated game interfaces.

The decision was supported by the team composition: the primary developer's existing expertise is in Kotlin for Android. KMP allows the same language and the same idioms to be extended to iOS rather than requiring the acquisition of a second language and platform toolchain. This pragmatic alignment of technology choice with team capability is itself an engineering consideration: a solution that the team can implement correctly is preferable to a theoretically superior solution that the team cannot.

2.4 AI-Generated Content in Education

2.4.1 Large Language Models for Educational Content

Large language models (LLMs) are AI systems trained on large corpora of text to predict and generate coherent language. Their application to educational content generation is a rapidly growing area of research and practice. LLMs can produce question-and-answer pairs, explanatory summaries, and scenario-based tasks at scale and at low marginal cost compared to human authoring, making them an attractive solution for the content authoring bottleneck identified in Chapter 1.

The primary engineering challenges when using LLMs for educational content generation are: factual accuracy, since models can produce plausible but incorrect statements; content diversity, since models tend to cluster around common framings of a topic; structural consistency, since models may produce responses that do not conform to the required schema; and deduplication, since models have no memory between invocations and will regenerate conceptually equivalent content given sufficiently similar prompts. Each of these challenges requires a deliberate pipeline design rather than naive invocation of the model API.

2.4.2 LangChain4j and the Mistral API

LangChain4j is a Java library that provides abstractions for building LLM-powered applications, including prompt templating, model chaining, output parsing, and schema validation [3]. It supports multiple model backends through a uniform interface, enabling the underlying model to be replaced without changing application code. U!REKA Play4Change uses LangChain4j to orchestrate nightly batch jobs

that generate daily tasks parameterised by subject domain and difficulty level, validate the structure of each response against a defined schema, and retry failed validations before discarding malformed outputs.

The Mistral API provides access to a family of instruction-tuned language models well suited to structured text generation tasks [4]. Instruction-tuned models are trained to follow explicit instructions embedded in the prompt rather than simply completing the text, making them more reliable for constrained generation tasks such as producing valid JSON-structured educational tasks. The combination of LangChain4j's structured output capabilities with Mistral's instruction-following behaviour provides the reliability required for an automated nightly pipeline.

2.4.3 Semantic Deduplication with pgvector

A fundamental property of language models is that they have no memory between separate invocations. A model asked today to generate a sustainability task and asked again tomorrow has no recollection of yesterday's output. As the task library grows, the probability that a newly generated task is conceptually equivalent to an existing one increases. Crucially, equivalent tasks are paraphrases, not duplicates: "sort your recycling correctly"and "separate your waste into the appropriate bins"convey the same educational action with no overlapping words. Exact text comparison cannot detect this equivalence.

The correct approach is to compare the meaning of texts rather than their words. Vector embeddings are numerical representations of text meaning: every piece of text is mapped to a point in a high-dimensional space, where texts with similar meanings are mapped to nearby points. A sentence embedding model — a model trained specifically to produce embeddings where semantic similarity corresponds to geometric proximity — converts each generated task into a vector before storage.

The geometric metric used for comparison matters. Euclidean distance measures the straight-line distance between two points in the embedding space. This distance is sensitive to the magnitude of the vectors, which is influenced by sentence length rather than meaning. Cosine similarity measures the angle between two vectors rather than the distance between their tips. Because the angle is independent of vector magnitude, it correctly captures semantic similarity regardless of sentence length differences. Cosine similarity is the standard metric for semantic text comparison and is the metric used by pgvector [5].

PostgreSQL extended with the pgvector module enables vector storage and cosine similarity queries natively, without a separate vector database system. This integration means that semantic deduplication queries and relational data queries share a single transactional context, eliminating consistency risks that arise when data is distributed across multiple storage systems. An approximate nearest-neighbour index reduces the query cost from linear in the number of stored tasks to approximately sub-linear, making the deduplication check tractable as the task library grows.

2.5 Related Platforms

Several gamified learning platforms have gained widespread adoption and serve as the primary reference points for this work.

Duolingo applies a streak-based engagement model to language learning and is the most studied platform for the long-term effectiveness of gamification [2]. Its retention mechanisms — streaks, leaderboards, and daily goals — are among the best validated in the literature. However, all of Duolingo's content is hand-authored by domain specialists. This model produces high-quality content but cannot be replicated by organisations without Duolingo's scale and budget. Duolingo is also a single-domain platform; its content model does not generalise. The platform delivers native Android and iOS applications built from separate native codebases.

Kahoot is a synchronous, instructor-led quiz platform designed for classroom use [2]. Players join a live session hosted by an instructor and answer questions on their own devices while results are displayed publicly. This model is effective for classroom engagement but is not suitable for self-paced asynchronous learning, which is the target use case for U!REKA Play4Change. Kahoot does not use AI-generated content.

Quizlet is a flashcard and study tool platform that allows communities to create and share content. It provides no scoring engine, no AI-generated content, and no streak or penalty mechanics. Its community content model distributes authoring effort across users but provides no quality control and no deduplication. Quizlet's mobile clients are built as separate native applications.

None of these platforms combines automated AI content generation with multiplatform delivery from a single shared codebase.

2.6 Identified Gap

The survey above reveals a consistent pattern: the platforms that have achieved proven engagement outcomes rely on manually authored content, and the platforms that support automated content generation do not apply the engagement mechanics and delivery infrastructure that sustain long-term learner motivation. Furthermore, no existing platform in this space is known to deliver both Android and iOS applications from a single shared codebase while also solving the AI content generation and deduplication problem.

U!REKA Play4Change addresses this gap by combining four capabilities that have not previously been integrated in a single platform: multiplatform delivery from a single KMP codebase; fully automated nightly AI content generation via LangChain4j and the Mistral API; semantic deduplication via pgvector to prevent content fatigue; and a server-side scoring engine with reward and penalty mechanics. Each of these capabilities exists independently in the literature and in commercial products. Their combination in a single deployable system is the novel contribution of this work.

Capítulo 3

Architecture and Design

This chapter describes the architecture of U!REKA Play4Change and the engineering reasoning behind each design decision. For every significant component, the chapter identifies the problem that required a decision, surveys the candidate approaches that exist to solve it, evaluates their trade-offs, and justifies the chosen approach. The goal is not to describe what was built, but to explain why it was built that way.

3.1 Architectural Overview

U!REKA Play4Change is composed of three principal components: a mobile client, a server-side backend, and a content generation pipeline. These components are connected by a set of design decisions that reflect the engineering constraints identified in Chapter 1: AI inference cannot be on the user's request path; player scores must be correct under concurrent access; the mobile application must function during network unavailability; and the cost model must not scale linearly with users.

The mobile client runs on Android and iOS from a single KMP (Kotlin Multiplatform) codebase. It communicates with the backend over HTTPS, caches task content locally, and queues score submissions in a local outbox when the network is unavailable. The backend is a stateless Spring Boot application that serves task content, records completions, and manages the scoring engine. A nightly batch job, executed within the same backend process, calls the Mistral language model API to generate new content, validates and deduplicates it, and writes valid tasks to the database. All persistent state is stored in a single PostgreSQL database extended with the pgvector module for vector similarity search.

3.2 Content Generation Pipeline

3.2.1 The Case Against On-Demand Generation

The platform's defining feature is AI-generated educational content. When designing how this content reaches a player, the most intuitive approach is to generate it when the player requests it: the player opens the app, a request goes to the server, the server calls the AI service, waits for a response, and returns the task. This is called synchronous on-demand generation.

This approach has three fatal properties for a mobile educational platform. First, language model inference is not fast. Generating a single task requires sending a prompt to the AI service, waiting for it to process the input, and receiving the output. Depending on model size and server load, this takes between two and eight seconds. A mobile user who opens an app and sees a loading screen for five seconds before content appears will stop opening the app. The engagement loop breaks at the first interaction.

Second, a platform using on-demand generation is available to users only if the AI service is available. Third-party services have outages, rate limits, and maintenance windows. If the AI service is unavailable when a user opens the app, the user sees an error. The platform's reliability becomes dependent on a component it does not control, available during every hour the platform is in use.

Third, cost scales linearly with users. Every user session triggers at least one API call to the AI service. As the user base grows, the daily API cost grows proportionally. A platform that succeeds becomes more expensive at the same rate it grows.

3.2.2 Batch Pre-Generation

The solution is to separate the moment of generation from the moment of delivery. Rather than generating content when users request it, a scheduled job generates all content for the following day during a low-traffic period at night, stores the results, and serves them from storage when users arrive. This is the batch pre-generation model.

Tabela 3.1: Comparison of content generation strategies.

Approach	Latency	Cost Model	AI Availability Dependency	Quality Gate
Synchronous on-demand	2–8 s	Per user	Coupled	None
Async with polling	Non-blocking	Per user	Coupled	None
Streaming	Perceived low	Per user	Coupled	None
Batch pre-generation	**Zero**	**Fixed**	**Decoupled**	**Full**

Table 3.1 compares the four candidate generation strategies. Batch pre-generation is the only approach that resolves all three problems simultaneously: zero latency on the user request path, fixed cost regardless of user count, and complete decoupling of user-facing availability from AI service availability.

The cost advantage is quantified as follows. In the on-demand model, the daily cost is proportional to the number of active users. In the batch model, the daily cost is proportional to the number of subject domains multiplied by the number of tasks per domain — a fixed quantity controlled by platform configuration, not by user behaviour:

$$\text{On-demand cost} = \text{active_users} \times \text{cost_per_generation} \tag{3.1}$$

$$\text{Batch cost} = \text{domains} \times \text{tasks_per_domain} \times \text{cost_per_generation} \tag{3.2}$$

At 1,000 active users and a configuration of 5 domains with 20 tasks each, the batch model costs approximately 10 times less than the on-demand model. At 10,000 users, the batch model costs approximately

100 times less. The cost advantage compounds with growth. The batch architecture is not merely a latency optimisation — it is the difference between a cost model that collapses under success and one that improves with it.

The key engineering insight is that changing *when* a problem is solved can change its cost structure entirely. By running generation at 23:00 each night rather than on each user's request, the failure surface shrinks from all hours of operation to a single known window each night, and fallback strategies for that window are tractable: the batch job retries with exponential delays; if all retries are exhausted, a curated static content library provides fallback content; the developer is alerted by 06:00 so the issue can be investigated before users are active.

The batch job is implemented as a Spring `@Scheduled` task within the backend process. It runs at a fixed time each night, generates tasks for each configured domain and difficulty level, and writes valid results to the database.

3.2.3 Semantic Deduplication

A language model has no memory between invocations. If the same prompt is issued on two consecutive nights, the model may produce tasks that are conceptually equivalent — the same educational idea expressed in different words. Over weeks and months, a player will notice that they are being asked to do the same thing repeatedly. The engagement loop deteriorates.

The naive solution is to check whether the exact text of a new task already exists in the database. This fails entirely because language models paraphrase naturally. The texts "sort your recycling correctly"and "separate your waste into the appropriate bins"have no words in common yet convey the same educational instruction. Exact text comparison cannot detect this equivalence.

The correct framing is to compare the meaning of texts rather than their words. Vector embeddings provide this capability. A sentence embedding model converts any piece of text into a high-dimensional numerical vector — an array of floating-point numbers — where texts with similar meanings produce vectors that point in similar directions in the embedding space.

Measuring similarity between two vectors requires choosing a metric. Euclidean distance measures the straight-line distance between the tips of the vectors. This is sensitive to vector magnitude, which depends on sentence length rather than semantic content. A long sentence and a short sentence expressing the same idea will have vectors of different magnitudes, and their Euclidean distance will be large despite identical meaning. Cosine similarity measures the angle between two vectors rather than the distance between their tips. The angle is independent of vector magnitude, so long and short sentences expressing the same idea produce a cosine similarity near 1.0 regardless of their length difference. Cosine similarity is the correct metric for semantic text comparison.

The deduplication procedure runs within the nightly batch job, before any new task is committed to the database: the new task is embedded, and its vector is compared against all stored task vectors using cosine similarity. If any stored task has a similarity above a defined threshold, the new task is discarded. A task below the threshold is stored along with its embedding vector for future comparisons.

With thousands of stored tasks, comparing a new vector against every stored vector in sequence becomes slow. An approximate nearest-neighbour index groups stored vectors into clusters; a new vector is compared only against the nearest cluster rather than every entry. This reduces query cost from O(n) to

approximately O($\sqrt{n}$), at the cost of occasionally missing the absolute nearest neighbour. For deduplication purposes, this approximation is acceptable: whether a task is 91% or 93% similar to an existing one, both would correctly be rejected by a threshold of 90%.

The pgvector extension for PostgreSQL implements both vector storage and approximate nearest-neighbour search natively within the relational database. This means deduplication queries execute within the same database transaction as the task insertion itself — if the deduplication check succeeds and the insertion fails, no orphaned vectors are created.

3.2.4 Content Quality Validation

Language models sometimes produce responses that do not conform to the expected structure. A task might be missing required fields, have a field of the wrong type, or contain a response that is incoherent. In a synchronous system, malformed responses can be retried immediately. In a batch system, malformed content must not silently enter the database and later be delivered to a player.

The pipeline validates every model response against a defined schema before any write occurs. If validation fails, the job retries the generation with the same parameters up to a fixed number of times. If all retries are exhausted, the task slot is discarded. A day with fewer tasks than the configured target is preferable to a day with one malformed task delivered to a player. The schema defines required fields, field types, and length constraints. Any response outside this definition is malformed and is not stored.

3.3 Data Persistence

3.3.1 Why a Single Relational Database

The platform stores several categories of data with different characteristics: player identity and credentials, which must be accurate and never lost; player scores and progress, which must be transactionally correct; task content and embeddings, which are written once and read many times; and semantic vectors, which require specialised query operations. A first architectural question is whether one storage system can serve all these needs, or whether each category should use a specialised system.

Using multiple storage systems introduces a synchronisation problem. If a task is written to the relational database but the write to a separate vector database fails, the system is inconsistent. Making such cross-system writes reliable requires distributed transaction coordination — a significantly more complex engineering problem than single-system transactions, with its own failure modes and operational burden.

A single relational database extended with vector capabilities avoids this problem. All data, including vectors, is stored in one system. Every write — task content, embedding vector, and any associated metadata — either all succeed or all fail within a single atomic transaction. There is no synchronisation problem because there is one system. The operational burden is that of one database, not two.

PostgreSQL with the pgvector extension satisfies both requirements: full relational capabilities with ACID (Atomicity, Consistency, Isolation, Durability) transaction guarantees, and native vector similarity search. This combination makes a single database the correct choice.

3.3.2 Transactional Correctness and ACID

ACID is a set of properties that a database must guarantee for transactions — groups of operations that should behave as a single unit. These properties are essential for correctness in a system where scores represent real player effort.

Atomicity means "all or nothing."When a player completes a task, two writes must happen together: the score is increased, and the task is marked as complete. If the server process crashes between these two writes, the player either has points without a completion record (which can be exploited) or a completion record without points (which is unfair). Atomicity guarantees that both writes succeed or neither does. This is analogous to a bank transfer: either both accounts are updated or neither is — there is no halfway state where money has left one account but not arrived at the other.

Isolation means that concurrent requests cannot interfere with each other's operations. If two requests for the same player's score arrive simultaneously and both read the current score before either writes, one write will overwrite the other, and the player will receive fewer points than earned. Isolation prevents this by ensuring that each transaction sees a consistent state of the data.

Durability means that a confirmed write survives a server crash. Once the database confirms that a score update has been committed, that update is present even if the server loses power immediately after. The player earned those points; the database's commitment is the system's commitment to the player.

3.3.3 Concurrent Write Safety

Multiple requests for the same player's data can arrive at the server simultaneously, particularly when the mobile client retries a submission after a network timeout. Without protection, two simultaneous score updates can interfere, producing a result where one update silently overwrites the other.

The problem is a race condition: Request A reads the score, Request B reads the same score, both compute an updated value, and one write overwrites the other. Neither write fails — both succeed silently — but the final score is incorrect.

Optimistic locking resolves this without serialising requests. Every score record carries a version counter. When a request reads the score, it notes the current version. When it writes the updated score, it includes a condition: only apply this update if the version is still the value read. If another request has already incremented the version, the condition fails, the write returns zero rows updated, and the application retries. No lock is held between the read and the write, so concurrent requests proceed freely and conflict resolution happens only in the rare case of an actual collision.

For simple score increments that do not require any conditional logic, the database can express the operation atomically: `score = score + 10` rather than read, compute, write. The database performs the read and write as a single atomic unit; no race window exists.

A separate idempotency problem arises from network retries. A player submits a task completion; the network drops before the acknowledgement arrives; the client retries. Without protection, the score is applied twice. A unique constraint at the database level on the combination of player identifier and task identifier prevents this: the second insert fails immediately with a constraint violation, which the application catches and interprets as a duplicate rather than an error. The player receives their points exactly once, regardless of how many times the client retried.

3.4 Mobile Client Architecture

3.4.1 Offline-First Design

Mobile networks are unreliable. Players use the platform in locations with poor or absent connectivity: public transport, basements, areas with limited signal. A platform that requires a live network connection for every interaction will fail users in these situations, breaking the engagement loop at the most inconvenient moments.

The offline-first principle inverts the assumption: the client should function correctly without a network connection, synchronising with the server whenever connectivity is available rather than requiring it for every operation. This means two things in practice.

First, task content is cached on the device after its first successful download. Subsequent accesses serve content from the local cache rather than from the network. A player who loaded today's task content while connected can still access it after losing connectivity. The cache is invalidated at midnight, when new content for the following day becomes available.

Second, score submissions use the outbox pattern. Before the client attempts to send a completion record to the server, it writes the intent to a local persistent queue — the outbox. A background process monitors the queue and attempts to flush each entry to the server. When the server confirms receipt, the entry is removed from the queue. If the application is closed or the network drops, the queue persists on disk. The next time the application has connectivity, the background process resumes flushing the queue. The server must handle duplicate submissions gracefully — the idempotency constraint described above ensures that a retried submission is recognised and ignored rather than applied twice.

3.4.2 Caching Strategy

Task content has a natural lifecycle: it is generated once per night, remains immutable throughout the day, and is replaced by new content at midnight. For immutable data with a predictable lifecycle, time-based cache expiry at the lifecycle boundary is the correct strategy. The cache holds task content until midnight; at midnight, the cache is invalidated and new content is fetched on the next access. There is no need for event-driven invalidation because the content does not change within its lifecycle.

Player scores require a different strategy. Scores change every time a task is completed or a penalty is applied. A cached score that is even one update behind is incorrect. For score data, the correct strategy is write-through caching: when a score is updated, the cache is updated in the same operation, before success is reported. The cache is never stale because it is updated atomically with the authoritative store.

When the cache reaches capacity, it must decide which entries to evict. LRU (Least Recently Used) eviction discards the entry that has not been accessed for the longest time. For this platform, whose users are enrolled in specific subject domains and access content within those domains repeatedly, LRU naturally keeps active domain content warm in the cache while allowing inactive domain content to fall out.

3.5 Authentication and Session Management

3.5.1 Delegated Authentication

Every player action — submitting a task completion, fetching a score, claiming a badge — must be attributed to a verified identity. The platform must know who each user is. The traditional approach is for the platform to manage passwords: users create accounts, the platform hashes and stores their passwords, and compares hashes on login. This approach places the responsibility for credential security on the platform. If the platform's database is breached, all user credentials are exposed. Users who reuse passwords across services have those other accounts compromised as a consequence.

Delegated authentication eliminates this risk by design. Instead of authenticating users itself, the platform relies on a trusted third-party identity provider that the user already has an account with. The user authenticates with the identity provider, which confirms their identity to the platform by issuing a cryptographic token. The platform never sees a password. There is no password database to breach.

This model follows the OAuth 2.0 protocol, the industry standard for delegated authorisation [6]. OAuth 2.0 defines a structured flow in which the identity provider and the platform exchange tokens with defined scopes and lifetimes, each step cryptographically verifiable.

After the identity provider confirms the user's identity, the backend issues its own JWT (JSON Web Token — a signed, self-contained credential) for subsequent requests. A JWT encodes the user's identity and any relevant permissions as a structured payload, signed with a secret key. Any backend server instance can verify the JWT by checking the signature without consulting a central session store, enabling the backend to scale horizontally without shared state.

3.5.2 Dual-Token Architecture

A JWT issued at login must have a lifetime. A long-lived token that remains valid for days is convenient but creates a large exposure window: a stolen token grants access for the token's full lifetime. A short-lived token that expires in minutes limits the exposure window but requires the user to re-authenticate frequently, disrupting the engagement experience.

The dual-token architecture resolves this tension. The backend issues two tokens at login. The access token is short-lived, expiring in 15 minutes. It is held only in application memory, never written to persistent storage, so it disappears when the application is closed or the process is terminated. The refresh token is long-lived, valid for 7 days, and stored in a secure, application-sandboxed location on the device. When the access token expires, the application silently presents the refresh token to receive a new access token — the user does not need to log in again.

Critically, when a refresh token is used, it is immediately invalidated and a new one is issued. The server records which refresh token is current for each session. If a stolen refresh token is used — by an attacker — the server issues a new token to the attacker. When the legitimate user next uses their own (now stale) refresh token, the server detects that a token has been presented after it was already invalidated. This is evidence of theft. The server invalidates all tokens for the account and forces re-authentication. The attack is detected and contained, even though the attacker had temporary access.

3.6 Scoring Engine

The scoring engine is the mechanism that converts task completion into the motivational signal sustaining daily engagement. It operates on the server side, not the client, for two reasons: server-side execution prevents score manipulation by modifying client code, and it ensures correctness under concurrent submissions.

The engine evaluates each task completion against a set of configurable rules. A correct and timely response earns a base reward. A response submitted within a streak — a defined number of consecutive days of engagement — earns a multiplied reward. Inactivity beyond a configurable threshold triggers a penalty, applied to the player's current score. Specific milestone scores trigger badge awards, which are recorded separately and displayed in the player's profile.

Every evaluation is expressed as a database transaction that applies the score change and records the completion atomically, using the optimistic locking and idempotency mechanisms described in Section 3.3.3. The engine produces a response that the mobile client uses to display immediate feedback — points earned, badges awarded, current streak — without any additional round-trips to the server.

3.7 Consistency Trade-offs

3.7.1 Task Content: Availability over Consistency

In any distributed system, when a network disruption prevents communication between components, a choice must be made. Either the system continues serving responses that may be slightly stale (favouring availability), or it refuses to serve responses until it can guarantee they reflect the latest state (favouring consistency). This mathematical constraint, known as the CAP theorem, means no system can simultaneously guarantee perfect consistency and perfect availability during a network partition.

The resolution is that different data in the same system can have different requirements, and designing for this is an act of engineering maturity. A blanket policy of "always consistent"means players see error screens when today's task content cannot be confirmed as current. A blanket policy of "always available"means player scores can be wrong.

Task content is a candidate for availability over consistency. Consider what staleness looks like: a player receives yesterday's task because today's has not yet propagated to the cache. The content is still a real educational task — valid, well-formed, appropriate for the domain. The engagement loop continues. Compare this with the alternative: the player opens the app and sees an error message because the backend cannot confirm the latest task content. The player who cannot engage today is less likely to return tomorrow. The disruption to the engagement loop is worse than the stale content. Therefore, task content is served from a cache that may be slightly stale, and this is correct by design.

3.7.2 Player Scores: Consistency over Availability

Player scores are a candidate for consistency over availability. Consider what a stale score looks like: a player completes a task, expects to see their score increase, and instead sees the same number they had before. Or worse, the displayed score does not reflect recent completions. The core contract of a

gamified system — effort produces visible reward — is broken. A player whose effort is not visibly rewarded will stop making the effort. The platform fails at its fundamental purpose.

A brief wait for a consistent score read is acceptable. An incorrect score is not. Therefore, score reads are served from the authoritative source, not from a potentially stale cache, and writes apply write-through cache updates to keep the cache current. The consistency requirement demands this even at a small cost to read performance.

3.8 Observability

A system running in production is opaque by default. Users experience problems they cannot articulate. Without instrumentation, the developer has no way to know which component failed, how often, or with what impact. Observability is the set of tools that make the system's internal state visible from its external outputs.

Average latency is an insufficient metric. If 95% of requests take 50 milliseconds and 5% take 1,050 milliseconds, the average is approximately 100 milliseconds — a figure that suggests acceptable performance while hiding that one in twenty users has a deeply broken experience. Percentile metrics expose the distribution: P50 (the median) shows typical performance, P95 shows the experience of the majority of users at the slow end, and P99 identifies tail behaviour affecting a smaller but non-zero population.

Structured logs outperform plain text for operational investigation. A plain text log line such as `"Generated task 4821 in 1840ms"` cannot be queried without parsing. A structured log entry encodes the same event as a queryable document: event type, task identifier, duration, domain, tokens consumed, and a trace identifier. The trace identifier connects every log entry produced during a single operation — a batch run, a user request — into a reconstructable sequence. Filtering by trace identifier shows exactly what happened, in order, for any specific operation.

The platform emits structured logs from all components with consistent field names. The nightly batch job logs each generation attempt, its outcome, and the duration. The API layer logs request timing, authentication outcome, and upstream database query durations. These logs are the primary diagnostic tool for investigating failures reported after the fact.

3.9 Deployment

The backend, database, and any auxiliary services are deployed as a set of coordinated containers using Docker Compose. A container is a self-contained runtime environment that packages the application and all its dependencies. Docker Compose orchestrates multiple containers — in this case, the Spring Boot backend and the PostgreSQL database — as a single deployable unit, with defined networking between them, shared configuration, and coordinated startup order.

This deployment model provides two important properties. First, reproducibility: the same Docker Compose configuration that runs on the developer's machine also runs in production. The risk of environment-specific failures is reduced by the fact that both environments are identical. Second, portability: the application can be moved to any host that supports Docker without modifying code or configuration.

The stateless design of the backend, described in Section 3.6, means that scaling from one backend instance to multiple requires only configuration changes to a reverse proxy, with no changes to the application code. The current deployment runs a single backend instance, which is appropriate for the initial deployment scale. The architecture supports horizontal scaling when load requires it.

Capítulo 4

Implementation

This chapter describes how the architectural decisions of Chapter 3 were realised in code. Where Chapter 3 answers *why* each component was designed as it was, this chapter answers *how* it was built. Sections that involve internal implementation details not fully confirmed at the time of writing use `\todo` annotations to indicate where further expansion is needed.

4.1 Monorepo Structure

The project is structured as a monorepo: a single version-controlled repository containing both the mobile client and the backend server. This structure reflects the tight coupling between the client and backend at the API boundary: when an API contract changes, both sides of the boundary must be updated together. A monorepo makes this coupling visible in the version history and prevents the client and server from diverging silently across separate repositories.

The repository is divided into two top-level modules: the KMP (Kotlin Multiplatform) mobile client and the Spring Boot backend. Each module has its own build configuration but shares the same root build tooling, enabling coordinated builds and cross-module dependency management.

Expand with directory tree listing showing module structure.

4.2 Kotlin Multiplatform Client

4.2.1 Shared Business Logic

The KMP module defines a `commonMain` source set that contains all code shared across Android and iOS. This includes the data models that represent tasks, player scores, and session state; the network clients that communicate with the backend API; the local persistence layer that implements the client-side cache and the outbox queue; and the scoring engine state management that interprets server responses and updates the UI state.

Code in `commonMain` is compiled to JVM bytecode for Android and to native machine code for iOS using the Kotlin Native compiler. Platform-specific source sets (`androidMain` and `iosMain`) contain only the code that cannot be shared: platform-specific file system paths, platform-specific secure storage access for the refresh token, and any platform API calls that have no cross-platform abstraction.

Expand with code listing showing a shared data model and

4.2.2 Compose Multiplatform UI

The user interface is built with Compose Multiplatform, a declarative UI framework that extends Jetpack Compose — the standard Android UI toolkit — to also target iOS. All screen definitions, composable functions, and UI state holders are written once in `commonMain` and rendered natively on each platform.

The declarative model means the UI is expressed as a function of state: given the current state of the application — loading, displaying a task, displaying a score update — the composable functions describe what should appear on screen. The Compose runtime manages the difference between the current and previous state and applies only the necessary updates to the rendered output. This model maps naturally to the gamified UI pattern, where score changes, badge awards, and streak updates must be animated in response to player actions.

Expand with screenshot and code listing of the daily task screen composable.

4.2.3 Navigation with Decompose

Navigation between screens is implemented using Decompose, a KMP-compatible navigation library that manages the back stack and screen lifecycle in a platform-independent way. Each screen is associated with a component — a class that holds the screen's state and logic. Components are created and destroyed by the navigation framework as the player moves between screens, and their state persists across configuration changes such as device rotation.

Decompose was selected over alternatives because it does not depend on platform-specific navigation APIs, making the navigation logic fully shareable between Android and iOS without platform-specific adaptations.

Expand with navigation graph diagram and component hierarchy.

4.2.4 Dependency Injection with Koin

Dependencies — network clients, repositories, use cases — are wired together using Koin, a lightweight dependency injection framework for Kotlin that supports KMP. Koin defines dependencies as modules: each module declares how its components are created and what their dependencies are. At runtime, Koin resolves the dependency graph and provides each component with the objects it needs.

Dependency injection was chosen over direct instantiation to make components testable in isolation: a network client can be replaced with a test double in unit tests without modifying the component under test. Koin was selected over alternatives because its API is idiomatic Kotlin and its KMP support is mature.

Expand with Koin module definition listing.

4.3 Backend: Spring Boot

4.3.1 REST API Design

The backend exposes a REST API over HTTPS. Each resource — tasks, completions, scores, sessions — is addressed by a URL, and operations on that resource are expressed using standard HTTP methods: GET to retrieve, POST to create, PATCH to update. The API follows the principle that each endpoint should perform exactly one semantically coherent action, with the request body defining the parameters of that action.

All API responses use a consistent envelope structure: a status field indicating success or failure, a data field containing the resource or result, and an errors field containing structured error information when the request cannot be processed. This consistency allows the mobile client to use a single response-handling layer for all API calls.

Authentication is enforced by a filter applied to all protected endpoints. The filter extracts the JWT from the HTTP `Authorization` header, verifies the signature cryptographically, extracts the user identity from the payload, and makes it available to the endpoint handler. No session store is consulted; verification is purely computational.

Expand with OpenAPI specification excerpt showing task and completion endpoints.

4.3.2 Nightly Batch Job

The nightly batch job is implemented as a Spring `@Scheduled` task. Spring's scheduling infrastructure invokes the annotated method at the configured time each night, running it on a dedicated thread separate from the request-handling threads. The job iterates over all configured domains and difficulty levels, invokes the LangChain4j generation pipeline for each, and writes the resulting valid tasks to the database.

The batch job is idempotent by design. If the job is interrupted and restarted, duplicate content that was generated in the first partial run is rejected by the semantic deduplication layer before insertion, so the job can be rerun safely without producing duplicate tasks.

Expand with code listing of the scheduled task method and retry logic.

4.3.3 Scoring Engine Implementation

The scoring engine is implemented as a service class that receives a task completion request, evaluates the applicable rules, and produces a score update transaction. The evaluation logic reads the player's current score, current streak count, and the set of tasks completed today, then applies the configured reward or penalty and determines whether a badge has been earned.

The score update is applied using optimistic locking: the player record is read with its current version, the new score is computed, and the update is written with a version check. If the version check fails, the operation is retried. The task completion record is inserted with a unique constraint check; a duplicate insertion is caught and treated as an idempotent success.

Expand with scoring rule configuration format and code listing of evaluation logic.

4.4 Content Generation Pipeline

4.4.1 LangChain4j Integration

LangChain4j is integrated as a dependency of the backend module. The integration defines an AI service interface — a Java interface annotated with LangChain4j annotations that specify the prompt template and the expected output type. LangChain4j generates an implementation of this interface at runtime, handling prompt construction, model invocation, response parsing, and retry on transient errors.

The prompt template parameterises the generation request by domain, difficulty level, and any additional context. The expected output type is a structured Java record matching the task schema, and LangChain4j uses the model's structured output capabilities to produce a response that can be deserialised directly without manual parsing.

Expand with AI service interface definition and prompt template

4.4.2 Mistral API Configuration

The Mistral API is configured as the model backend for LangChain4j. The configuration specifies the model identifier, API key, and generation parameters: temperature (which controls the randomness of the output, set to a moderate value to balance diversity and coherence), maximum token count (set to constrain response length), and response format (JSON, to support structured output parsing).

The API key is provided as an environment variable and is never stored in the codebase or version control history.

Expand with configuration file excerpt and environment variable setup.

4.4.3 Schema Validation and Retry Logic

Every model response is validated against the task schema before any write occurs. The schema defines required fields, their types, and constraints on their values. Validation is implemented as a check on the deserialised output object.

If validation fails, the pipeline retries the generation call up to a configured maximum number of attempts. Each retry uses the same prompt; the non-determinism of the language model means subsequent attempts may produce a valid response even if the first attempt did not. If all retries are exhausted, the task slot is logged as failed and the pipeline moves on to the next slot. The batch job reports the total number of generated and failed slots at completion.

Expand with validation logic code listing and retry configuration.

4.4.4 pgvector Deduplication

After a task passes schema validation, its text is passed to a sentence embedding model to produce a vector representation of its meaning. The embedding model is invoked through LangChain4j's embedding API, which supports multiple backend models through the same interface.

The resulting vector is used in a pgvector similarity query against the stored task vectors. The query retrieves the most similar stored task and its cosine similarity score. If the similarity score exceeds the configured threshold, the new task is discarded as a near-duplicate. If it is below the threshold, the task and its vector are inserted together in a single database transaction.

Expand with SQL query listing for vector similarity check and insertion.

4.5 Database Schema

The database schema defines tables for player identity, player scores and progress, task content, task embeddings, and task completion records. The task content table is partitioned by date to ensure that queries for today's tasks touch only the current partition, keeping query performance predictable as the task library grows.

The player score table carries a version column for optimistic locking. The task completion table carries a unique constraint on the composite key of player identifier and task identifier, enforcing idempotency at the database level.

The pgvector extension is enabled with the `CREATE EXTENSION vector` command, and the embedding column in the task table is defined as type `vector(n)` where `n` is the dimensionality of the chosen embedding model.

Expand with full schema definition including all tables, indi-

4.6 Authentication Implementation

OAuth 2.0 delegated authentication is implemented using Spring Security's OAuth 2.0 client support. The application registers one or more identity providers. When a player initiates login, Spring Security redirects to the identity provider's authorisation endpoint. After the player authenticates, the identity provider redirects back to the application with an authorisation code. The application exchanges the code for an identity token, extracts the user's identity claims, and creates or retrieves the local player record.

JWT issuance is implemented using a JWT library. The backend signs the access token with a private key using the RS256 algorithm (RSA signature with SHA-256). The public key is available for verification by any backend instance, enabling stateless authentication across horizontally scaled instances. The refresh token is a randomly generated opaque string stored in the database with an associated expiry time and revocation flag. On every use, the refresh token is immediately revoked and a new one is issued, implementing the rotation described in Section 3.5.2.

Expand with Spring Security configuration and JWT issuance code listing.

4.7 Deployment with Docker Compose

The Docker Compose configuration defines two services: the Spring Boot backend and the PostgreSQL database. The database service uses an official PostgreSQL image with the pgvector extension pre-installed. The backend service is built from a multi-stage Dockerfile: the first stage compiles the application using a Gradle build, and the second stage copies the compiled jar into a minimal JRE image, keeping the final image as small as possible.

Environment variables provide all configuration that varies between environments: database credentials, API keys, signing keys, and scheduled job times. These variables are supplied at deployment time and are not stored in the Compose file itself.

The database service uses a named volume for persistent storage, ensuring that data survives container restarts. The backend service depends on the database service and will not start until the database is ready to accept connections.

Expand with Docker Compose file listing and deployment steps.

Capítulo 5

Evaluation

This chapter evaluates the platform against the four objectives stated in Chapter 1. It covers functional testing of the primary user and system flows, performance measurements of the batch pipeline and API layer, validation of the cost model, and an honest assessment of the platform's current limitations.

5.1 Functional Testing

Functional testing verified that each primary flow in the platform behaves correctly under normal conditions and under defined error conditions. The tested flows are as follows.

The authentication flow verifies that a player can authenticate via the configured OAuth 2.0 identity provider, that the backend issues a correctly signed access token and a refresh token, that the access token can be used to access protected endpoints, that an expired access token is rejected with a 401 response, that presenting a valid refresh token produces a new access token and a new refresh token, and that presenting a previously used refresh token triggers full session revocation.

The task completion flow verifies that a player can fetch today's tasks, submit a correct response, receive the expected score update, and see the completion reflected in their progress. It also verifies that submitting the same completion twice — simulating a network retry — results in a single score update, validating the idempotency constraint.

The score calculation flow verifies that the scoring engine applies the configured reward for a correct timely response, applies the streak multiplier when applicable, and applies the inactivity penalty after the configured threshold has passed without a completion.

The deduplication pipeline verifies that two tasks generated from the same domain on the same night that are semantically equivalent — as determined by cosine similarity exceeding the configured threshold — result in only one task being stored.

The offline submission flow verifies that a task completion written to the local outbox while the network is unavailable is delivered to the server when connectivity is restored, and that the resulting score update is applied correctly.

Expand with test results table showing pass/fail status for each tes-

5.2 Performance Evaluation

5.2.1 Batch Generation Latency

The nightly batch job must complete before users become active in the morning. The job generates all tasks for the following day, validates their structure, deduplicates them against stored vectors, and writes valid tasks to the database. The total duration of the job depends on the number of tasks generated, the response time of the Mistral API, and the duration of vector similarity queries.

The job must complete before 06:00 to ensure content is available when users begin their day. The measurement of interest is the end-to-end duration of the complete batch run across all configured domains and difficulty levels.

Expand with measured batch job duration across multiple runs, broken down by phase (generation, validation, deduplication, write), with P50 and P95 reported.

5.2.2 API Response Latency

Task content served from the server-side cache does not involve any database query or AI inference. The response latency for a task fetch request on a warm cache should be dominated by network round-trip time and serialisation cost, with no upstream processing time.

The measurements of interest are P50, P95, and P99 latency for the task fetch endpoint under typical load, and the same measurements for the score submission endpoint, which involves a database write and the scoring engine evaluation.

Expand with latency measurements from load testing, showing percentile distribution for the primary endpoints. Target: task fetch P95 under 100ms, score submission P95 under 200ms.

5.2.3 Cold-Start Behaviour

On the first launch of the application after installation, no content is cached locally. The player must wait for the initial task content to be fetched from the server. The measurement of interest is the time from first application launch to task content being displayed.

The cold-start latency is bounded by the network round-trip time to the server and the server's response time on a warm cache. Because content is pre-generated and cached, the server response time is low. The dominant factor on a typical mobile network is the round-trip time.

5.3 Cost Model Validation

The architectural decision to use batch pre-generation rather than on-demand generation was motivated in part by its cost model. Section 3.2.2 expressed the cost comparison algebraically. This section validates the model with concrete numbers.

Table 5.1 compares the daily API cost of the on-demand and batch models at three user scales, assuming a fixed configuration of 5 subject domains with 20 tasks per domain (100 total tasks per batch run), and a cost per generation of c (a placeholder for the actual Mistral API pricing):

Table 5.1 shows that the batch model breaks even with the on-demand model at exactly the number of tasks generated per batch run. Below this user count, both models cost similarly. Above this user count, the batch model becomes progressively cheaper. At 10,000 active users, the batch model costs 100 times

Expand with measured

Tabela 5.1: Daily AI generation cost comparison: batch versus on-demand.

Active Users	On-Demand Daily Cost	Batch Daily Cost
100	$100c$	$100c$ (break-even)
1,000	$1{,}000c$	$100c$ ($10\times$ cheaper)
10,000	$10{,}000c$	$100c$ ($100\times$ cheaper)

less per day than the on-demand model with the same content volume, as derived from Equations 3.1 in Chapter 3.

The batch model's cost does not change when users are added. Hiring, marketing, and institutional partnerships that grow the user base do not trigger proportionally higher infrastructure costs. This property makes the platform economically viable as a product for educational institutions of varying size.

Expand with actual measured API costs from the implementation's nightly runs, replacing the symbolic c with real figures.

5.4 Limitations and Future Work

5.4.1 Absence of Per-User Personalisation

The current design generates content at the domain level, not the user level. Every player enrolled in the same domain receives the same task on the same day. This is a deliberate simplification that preserves the fixed cost model: generating one set of tasks per domain, regardless of user count, is the source of the cost advantage described above.

The limitation is that the platform cannot adapt to a player's demonstrated strengths and weaknesses. A player who consistently answers questions about energy efficiency correctly would benefit from harder energy questions or questions in adjacent topics, but the current engine does not make this adaptation. Real-time per-user personalisation would require on-demand generation (reintroducing latency and per-user cost) or a hybrid model in which a larger pool is generated per batch and the server selects from the pool based on the player's history. The hybrid approach is technically tractable and is identified as future work.

5.4.2 iOS Deployment

Delivering a production iOS application requires an Apple Developer Program membership and submission through the App Store review process. These are extended objectives for this project. The KMP and Compose Multiplatform implementation targets iOS, and the codebase compiles for the iOS target. However, distribution through the App Store and thorough testing on physical iOS hardware remain as future work beyond the project's primary scope.

5.4.3 Observability Dashboard

The platform emits structured logs as described in Section 3.9. An extended objective is to aggregate these logs into a Prometheus metrics store and visualise them in a Grafana dashboard, providing real-time visibility into batch job outcomes, API latency distributions, and error rates. The infrastructure for

this observability layer exists and is configured in the Docker Compose file, but the dashboard definitions and alerting rules are not part of the core deliverable.

Capítulo 6

Conclusion

6.1 Summary of Contributions

U!REKA Play4Change makes four engineering contributions that, in combination, address a gap not filled by any existing platform in its domain.

The first contribution is a working multiplatform gamified educational platform delivered from a single KMP (Kotlin Multiplatform) codebase. The same shared business logic and Compose Multiplatform user interface code runs on both Android and iOS, producing two deployable applications without duplicating engineering effort. This demonstrates that KMP is a viable foundation for production gamified educational software at this scope.

The second contribution is a nightly AI content generation pipeline with semantic deduplication. The pipeline invokes the Mistral language model API through LangChain4j on a fixed nightly schedule, validates the structure of every generated task, and filters out semantically equivalent content using cosine similarity search over vector embeddings stored in PostgreSQL with pgvector. The pipeline operates at fixed daily cost regardless of user count and produces content that is ready before users become active, with zero inference latency on the user-facing request path.

The third contribution is a server-side scoring engine that sustains the daily engagement loop. The engine applies configurable rewards for timely correct completions, streak multipliers for consecutive days of engagement, penalties for inactivity, and badge awards for milestone scores. The engine is implemented with optimistic locking and idempotency constraints that guarantee correctness under concurrent submissions and network retries.

The fourth contribution is a cost model that scales sublinearly with users. By separating content generation from content delivery and generating content in fixed batches rather than on demand, the platform's primary operational cost — AI model inference — does not grow when users are added. This cost model is validated analytically and represents a structural advantage over on-demand generation that compounds with growth.

6.2 Objectives Achieved

Objective 1: Multiplatform delivery from a single codebase. This objective is met. The project produces an Android application and targets iOS from a single KMP project. The shared codebase includes

all business logic, data models, network clients, local persistence, and the Compose Multiplatform user interface. The iOS target compiles successfully; production distribution via the App Store is an extended objective beyond this project's primary scope.

Objective 2: Adaptive AI-generated content with semantic deduplication. This objective is met. The nightly batch pipeline generates domain-specific educational tasks using LangChain4j and the Mistral API, validates their structure, and discards semantically duplicate content using pgvector cosine similarity search. Content is available before users are active, with the deduplication threshold configurable to balance diversity and content volume.

Objective 3: Engagement loop engineering. This objective is met. The scoring engine is implemented on the server, evaluates configurable reward and penalty rules, applies streak multipliers, and issues badge awards. The engagement loop — open the app, complete a task, receive a score update, return tomorrow to maintain the streak — is fully functional. The correctness of the engine under concurrent access and network retries is guaranteed by optimistic locking and database-level idempotency constraints.

Objective 4: Cost efficiency through batch pre-generation. This objective is met analytically and validated in principle. The batch cost model is derived from first principles in Chapter 3 and compared quantitatively with the on-demand model in Chapter 5. At 10,000 active users with the current batch configuration, the batch model produces content at approximately 100 times lower cost per day than the on-demand alternative. Validation with real production traffic remains as extended future work.

6.3 Reflection on Engineering Decisions

The process of building U!REKA Play4Change required making explicit decisions under uncertainty, justifying each choice against the problem it was meant to solve rather than against technology preference. Several decisions produced insights that generalise beyond this specific project.

The most significant insight came from the content generation architecture. The first impulse when building a feature is to implement it where it is needed: if users need tasks, generate tasks when users ask. Changing this to batch pre-generation required accepting an inversion of the usual request-response model, but the inversion resolved four problems simultaneously: latency, cost, availability dependency, and the inability to inspect content before delivery. The insight is that changing *when* a problem is solved can change its entire cost structure. A problem that is expensive to solve at runtime may be cheap to solve in advance. This generalisation applies to many engineering problems: pre-computation, caching, and asynchronous processing all exploit this same principle.

The consistency trade-off decision demonstrated a different kind of insight: that a single platform can legitimately have different correctness requirements for different data, and that recognising this prevents over-engineering in one direction or under-engineering in another. Applying a uniform "always consistent"policy would have forced task content reads to wait for authoritative confirmation, breaking the engagement loop during network disruptions. Applying a uniform "always available"policy would have allowed score reads from stale replicas, breaking the core contract of the gamified system. Recognising that task content and player scores have genuinely different requirements, and designing accordingly, is an example of engineering maturity over one-size-fits-all solutions.

The dual-token authentication architecture illustrated the value of resolving an apparent dilemma by adding a dimension. The choice between long-lived tokens (convenient but risky) and short-lived tokens (safe but inconvenient) appeared binary. Adding a second token with a different lifetime and a rotation protocol resolved both concerns simultaneously, without compromising either. Many apparent binary engineering choices are resolvable by recognising that the two dimensions of the trade-off can be addressed by separate mechanisms.

6.4 Future Work

Three directions for future development are identified based on the limitations described in Chapter 5.

The first is per-user content personalisation without sacrificing the fixed cost model. The current design generates one set of tasks per domain per day, serving the same content to all players in that domain. A hybrid model would generate a larger pool of tasks per batch run and implement server-side selection logic that chooses from the pool based on each player's completion history, demonstrated strengths, and identified knowledge gaps. This preserves the batch cost structure while enabling adaptive content delivery. The selection logic would run at query time, using the player's completion records to filter and rank the available pool.

The second is full production deployment of the iOS application through the Apple App Store. The KMP codebase targets iOS and the application builds for the iOS platform. The remaining work involves App Store submission, compliance with Apple's review guidelines, and thorough testing on physical iOS hardware across a range of devices and iOS versions.

The third is a production observability dashboard built on the structured logging infrastructure already in place. The platform emits structured JSON logs from all components. Aggregating these into a Prometheus metrics store and building a Grafana dashboard would provide real-time visibility into batch job health, API latency distributions, error rates, and daily content generation success rates. Alert rules on the P95 batch completion time and the API error rate would enable proactive incident response rather than reactive investigation.

Referências

[1] Sebastian Deterding, Dan Dixon, Rilla Khaled & Lennart Nacke, "From Game Design Elements to Gamefulness: Defining Gamification", em *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, sér. MindTrek '11, New York, NY, USA: ACM, 2011, páginas 9–15.

[2] Juho Hamari, Jonna Koivisto & Harri Sarsa, "Does Gamification Work? – A Literature Review of Empirical Studies on Gamification", em *Proceedings of the 47th Hawaii International Conference on System Sciences*, sér. HICSS '14, Washington, DC, USA: IEEE, 2014, páginas 3025–3034.

[3] LangChain4j Contributors, *LangChain4j – Java Implementation of LangChain*, https://langchain4j.dev, Accessed: March 2026.

[4] Mistral AI, *Mistral AI Models*, https://mistral.ai, Accessed: March 2026.

[5] pgvector Contributors, *pgvector: Open-Source Vector Similarity Search for Postgres*, https://github.com/pgvector/pgvector, Accessed: March 2026.

[6] Pivotal Software, *Spring Boot Reference Documentation*, https://docs.spring.io/spring-boot/docs/current/reference/html/, Accessed: March 2026.

[7] Yu Zhonggen, "A Meta-Analysis of Use of Serious Games in Education over a Decade", *International Journal of Computer Games Technology*, vol. 2019, páginas 1–8, 2019.

Apêndice A

Applied Survival Analysis by Hosmer and Lemeshow

Stata Textbook Examples Applied Survival Analysis by Hosmer and Lemeshow [**newtest**]

The data files used for the examples in this text can be downloaded in a zip file from the Wiley FTP website or the Stata Web site.

```
# The R package(s) needed for this chapter is the survival package.
# We currently use R 2.0.1 patched version. You may want to make sure
# that packages on your local machine are up to date. You can perform
# updating in R using update.packages() function.

# url: http://www.ats.ucla.edu/stat/r/examples/
# data set is hmohiv.csv.
hmohiv<-read.table("http://www.ats.ucla.edu/stat/r/examples/asa/hmohiv.csv", sep=",",
    header = TRUE)
attach(hmohiv)
hmohiv

# using the hmohiv data set. To control the type of symbol, a variable called psymbol is created.
# It takes value 1 and 2, so the symbol type will be 1 and 2.
psymbol<-censor+1
table(psymbol)

plot(age, time, pch=(psymbol))
legend(40, 60, c("Censor=1", "Censor=0"), pch=(psymbol))

age1<-1000/age
plot(age1, time, pch=(psymbol))
legend(40, 30, c("Censor=1", "Censor=0"), pch=(psymbol))

# Package "survival" is needed for this analysis and for most of the analyses in the book.
library(survival)
test <- survreg( Surv(time, censor) ~ age, dist="exponential")
```

```
summary(test)

pred <- predict(test, type="response")
ord<-order(age)
age_ord<-age[ord]
pred_ord<-pred[ord]
plot(age, time, pch=(psymbol))
lines(age_ord, pred_ord)
legend(40, 60, c("Censor=1", "Censor=0"), pch=(psymbol))
```